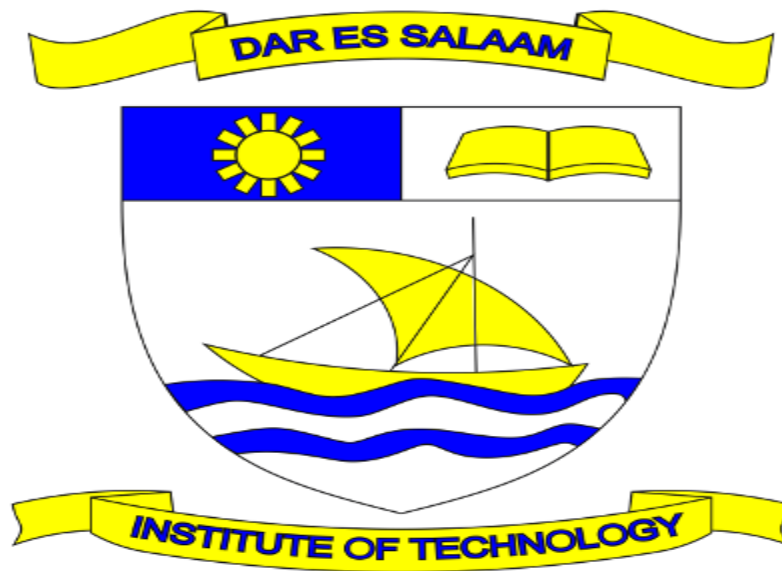




Name: \_\_MSHAM, KHASIM RAJABU.

Course: Fundamentals of Data Structures and Algorithms

Assignment 01

Due: June 03, 2026

**1. Rank the following complexities in terms of their rate of growth starting with slowest to fastest:**

$O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ ,  $O(n^5)$

**2. Find the Big-O complexity for the following functions:**

a)  $4n^2 + 20n + 9$

The dominant term is  $4n^2$ . Drop the constant and lower-order terms.

Big-O:  $O(n^2)$

b)  $25n^3 + 15$

The dominant term is  $25n^3$ .

Big-O:  $O(n^3)$

c)  $n^3 + 5 \log n + 6$

The dominant term is  $n^3$ .

Big-O:  $O(n^3)$

d)  $10^5 + 10^5n + 10^5n^2$

The dominant term is  $10^5n^2$ . Drop lower-order terms and constants.

Big-O:  $O(n^2)$

e)  $72n(n + 3n^2)$

Expand:  $72n \times n + 72n \times 3n^2 = 72n^2 + 216n^3$

The dominant term is  $216n^3$ .

Big-O:  $O(n^3)$

f)  $2^8 + 324n - 5$

The dominant term is  $324n$ . The  $2^8$  and  $-5$  are constants.

Big-O:  $O(n)$

**3. Design an algorithm that runs in worst-case  $O(n)$  time to add all the numbers from 1 to  $n$  (including  $n$ ).**

The algorithm below uses a single loop that runs  $n$  times with an  $O(1)$  body, so the overall complexity is  $O(n)$ .

```
sum := 0
```

```
for i := 1 to n do:
```

```
    sum := sum + i
```

Note: There is also a formula  $\text{sum} = n(n+1)/2$  that runs in  $O(1)$ , but this algorithm satisfies the  $O(n)$  requirement.

**4. For each of the following code fragments, give the worst-case running time complexity in Big-O notation:**

**a)**

$x = 4$  is  $O(1)$ .

The for loop runs 'number' iterations with an  $O(1)$  body.

Total complexity:  $O(1 + \text{number}) = O(\text{number})$

**b)**

`get_choice()` is  $O(1)$  by assumption.

The worst case is when `choice = 2`, which triggers the for loop.

The for loop runs 'cycles' iterations with an  $O(1)$  body.

Total worst-case complexity:  $O(\text{cycles})$

**c)**

First, find the complexity of `increase (val, iter)`:

- It has one for loop running `iter` times with  $O(1)$  body.
- So `increase` is  $O(\text{iter})$ .

Now for `calculate (val, iter)`:

- The outer loop runs `iter` times.
- Each iteration calls `increase (val, iter)` which costs  $O(\text{iter})$ .
- Total =  $\text{iter} \times O(\text{iter}) = O(\text{iter} \times \text{iter}) = O(\text{iter}^2)$

Complexity of `calculate`:  $O(\text{iter}^2)$

**d)**

`increase (val, iter)` is still  $O(\text{iter})$  as above.

In calculate, the outer loop runs iter times, but each call is increase (val, i) not increase (val, iter).

So on iteration i, the cost is  $O(i)$ .

Total =  $O(1) + O(2) + O(3) + \dots + O(\text{iter})$

$$= O(1 + 2 + 3 + \dots + \text{iter})$$

$$= O(\text{iter}(\text{iter}+1)/2)$$

$$= O(\text{iter}^2)$$

Complexity of calculate:  $O(\text{iter}^2)$

**5. Fill in the blanks with either  $\leq$ ,  $\geq$ , or  $=$**

(i)  $f(n)$  is  $O(g(n)) \Rightarrow f(n) \leq c g(n)$  for all  $n \geq k$

(ii)  $f(n)$  is  $\Omega(g(n)) \Rightarrow f(n) \geq c g(n)$  for all  $n \geq k$

**6. Show that  $4n + 12$  is  $O(n)$ .**

We need to find constants  $c$  and  $k$  such that  $4n + 12 \leq c \cdot n$  for all  $n \geq k$ .

Let  $c = 5$  and  $k = 12$ .

We need:  $4n + 12 \leq 5n$

$$12 \leq 5n - 4n$$

$$12 \leq n$$

This is true for all  $n \geq 12$ .

Therefore  $4n + 12 \leq 5 \cdot n$  for all  $n \geq 12$ , so  $4n + 12$  is  $O(n)$ .

**7. Show that  $5n^2 - 16$  is  $O(n^2)$ .**

**7. Show that  $5n^2 - 16$  is  $O(n^2)$ .**

We need to find constants  $c$  and  $k$  such that  $5n^2 - 16 \leq c \cdot n^2$  for all  $n \geq k$ .

Let  $c = 5$ .

We need:  $5n^2 - 16 \leq 5n^2$

$$-16 \leq 0$$

This is always true for any value of  $n$ .

So let  $k = 1$ .

Therefore  $5n^2 - 16 \leq 5 \cdot n^2$  for all  $n \geq 1$ , so  $5n^2 - 16$  is  $O(n^2)$ .